

Simulateur de Circuits Électroniques

Documentation des programmes — Phase 1 terminée

STI2D · ngspice · Arduino UNO · Interface graphique Canvas
Document unique — sommaire et rôle de chaque module

Généré le 18 juin 2026

Sommaire

1. 1. Vue d'ensemble et flux de données
2. 2. Interface utilisateur (Simulateur/*.js)
 - [app.js](#) · [state.js](#) · [render.js](#) · [geometry.js](#)
 - [theme.js](#) · [modal-ui.js](#) · [value-prompt.js](#) · [json-utils.js](#)
 - Layouts composants (CD4511, HC90, UNO, Grove, bargraph...)
 - Panneaux et popups (scope, Bode, série, sources, Arduino)
 - [index.html](#) · [style.css](#)
3. 3. Simulation et animation
 - [simulation.js](#) · [led-animation.js](#) · [scope-animation.js](#) · [speaker-audio.js](#)
4. 4. Moteur SPICE (Simulateur/Engine/)
 - [schematic-to-spice.mjs](#) · [spice-netlist-v2.mjs](#) · [v2/result-parser.mjs](#)
 - Logique numérique (portes, CD4511, 74HC90, XSPICE...)
 - Instruments, BCD, voltmètre, I²C
5. 5. Co-simulation Arduino
6. 6. Serveur et déploiement
7. 7. Outils Node.js (tools/)

1. Vue d'ensemble et flux de données

Chaîne principale :

L'utilisateur édite le schéma dans le navigateur (`app.js`, `render.js`) → clic « Lancer Simulation » (`simulation.js`) → requête POST `/api/simulate` → le serveur convertit le JSON en netlist ngspice (`schematic-to-spice.mjs`, `spice-netlist-v2.mjs`) → exécution ngspice → parsing des résultats (`result-parser.mjs`) → retour JSON → animations LED, oscilloscope, bargraph, 7 segments, audio (`led-animation.js`, `scope-animation.js`, `speaker-audio.js`).

Arduino : les sketches sont interprétés localement (`arduino-sketch-parse.mjs`) pour piloter GPIO, LCD Grove, DHT22 et moniteur série sans attendre SPICE lorsque c'est possible.

2. Interface utilisateur

Modules JavaScript du dossier *Simulateur/* — éditeur graphique, menus, panneaux et rendu Canvas.

app.js Interface · Point d'entrée

Fichier principal du front-end. Initialise l'éditeur : glisser-déposer de composants, câblage, menus Fichier/Éditeur/Sources/Composants/Logique/Mesures, raccourcis clavier, undo/redo, sauvegarde et ouverture de circuits JSON, double-clic sur composants (valeurs, éditeur Arduino, documentation), lancement et arrêt de la simulation.

state.js Interface · État global

État partagé : référence canvas/contexte, zoom et pan, grille (GRID_SIZE), drapeaux (flags.isSimulating), circuit (components, wires, autoJunctions), sélection, résultats SPICE (simulationResults), compteurs de labels auto, piles undo/redo.

Exports : canvas, ctx, GRID_SIZE, scale, pan, flags, circuit, simulationResults, saveState, snapToGrid, toGridCoords...

renderer.js Interface · Rendu

Dessin Canvas 2D de tous les composants (passifs, logique, UNO, LCD Grove, DHT22, afficheurs 7 segments, bargraph DC10H, voltmètres, ampèremètres, oscilloscope...). Affiche les valeurs simulées, états animés (LED, segments, bargraph) et surbrillance de sélection.

Exports : resizeCanvas, draw

geometry.js Interface · Géométrie

Calculs géométriques et interactions : collision souris/composant, détection de fils et intersections, jonctions par type de boîtier, contrôles potentiometre/interrupteur/bouton, tension idéale à une jonction pour affichage.

Exports : componentHitTest, getComponentJunctions, getVoltageAtJunction, potentiometerControlHit...

theme.js Interface · Thème

Gestion du thème sombre/clair de l'éditeur avec palettes de couleurs (canvas, fils, composants) persistées dans localStorage.

Exports : COLORS, setEditorTheme, initEditorTheme

modal-ui.js Interface · Modales

Affichage/masquage des fenêtres modales CSS sans bloquer la barre de menus (compatible Electron Simulateur H).

Exports : showModal, hideModal, initModalUi

value-prompt.js Interface · Saisie

Boîte de dialogue modale de saisie (remplace window.prompt, compatible Electron).

Exports : showValuePrompt, initValuePrompt

json-utils.js Interface · Fichiers

Parse et normalise les fichiers circuit JSON (UTF-8, validation structure, migration des composants à l'ouverture).

Exports : `parseJsonText`, `normalizeCircuitPayload`, `migrateLoadedComponents`

Layouts des composants (géométrie UI et brochage)

cd4511-layout.js Layout

Boîtier DIP CD4511 : positions des broches BCD (A–D), segments a–g, LT/BI, mapping jonction → clé terminal SPICE.

ic74hc90-layout.js Layout

Boîtier DIP 74HC90/74LS90 : brochage 14 broches, sorties Q0–Q3, mapping jonction → terminal.

arduino-uno-layout.js Layout

Carte Arduino UNO R3 : headers analogique/numérique, indices de broches, sketch par défaut, mapping jonctions.

Exports : `DEFAULT_ARDUINO_SKETCH`, `arduinoUnoJonctionToTerminalKey`

grove-lcd-layout.js Layout

Module Grove LCD 16×2 I²C : broches SDA/SCL/VCC/GND, zone d'écran, dimensions pour le hit-test.

dht22-layout.js Layout

Capteur Grove DHT22 : broches DATA/VCC/NC/GND et zone de rendu du capteur.

bargraph-dc10h-layout.js Layout

Bargraph DC10H : 10 segments LED, broche COM, palette de couleurs, hit-test par segment.

hd44780-font.js Layout · Police

Police bitmap HD44780 5×8 pour afficher les caractères sur l'écran LCD Grove simulé.

Exports : `getHd44780Glyph`

Panneaux, popups et éditeur Arduino

source-panel.js Panneau

Panneau bas de réglage des générateurs (GImp, GSin, GSqr) avec relance de simulation live à chaque modification.

gimp-panel.js Panneau

Réexporte l'API de `source-panel.js` sous les noms historiques « GImp ».

scope-panel.js Panneau

Panneau bas de l'oscilloscope : réglages time/div, volts/div, position CH1/CH2, ouverture du popup flottant.

scope-popup.js Popup

Fenêtre oscilloscope flottante déplaçable : courbes CH1/CH2, zoom temporel et décalage.

bode-popup.js Popup

Analyseur de Bode : courbes gain (dB) et phase en fonction de la fréquence.

serial-monitor-popup.js Popup

Moniteur série virtuel pour la sortie UART (Serial.print) des cartes Arduino UNO simulées.

arduino-editor.js Arduino · UI

Panneau latéral d'édition de sketch Arduino : compilation via `arduino-cli` (API REST), gestion des bibliothèques, synchronisation avant simulation.

Exports : `openArduinoEditor`, `compileActiveSketch`, `prepareArduinoForSimulation`, `initArduinoEditor`

arduino-lib-popup.js Arduino · UI

Popup d'aperçu et documentation des bibliothèques Arduino installées.

arduino-syntax-highlight.js Arduino · UI

Coloration syntaxique du code Arduino dans l'éditeur (mots-clés, types, commentaires).

arduino-sketch-sync.js Arduino · UI

Synchronise le texte de l'éditeur vers les objets `arduino_uno` du circuit avant simulation ou sauvegarde.

arduino-lib-commands.mjs Arduino · Référence

Catalogue des commandes par bibliothèque Arduino (LiquidCrystal I²C, DHT, Wire...) avec indication du support simulateur (`simSupported`).

Exports : `ARDUINO_LIB_DOCS`

Structure HTML et styles

index.html Interface · Structure

Page principale : barre de menus (Fichier, Éditeur, Sources, Composants, Logique, Mesures, Arduino), canvas, barre de simulation, modales, chargement ES modules.

style.css Interface · Styles

Feuille de styles complète : thème sombre/clair, menus déroulants, composants sur canvas, panneaux bas, popups scope/Bode/série, éditeur Arduino.

3. Simulation et animation

simulation.js Animation · Orchestrateur

Orchestre la simulation : construction de l'état JSON, POST vers `/api/simulate`, gestion des erreurs réseau, relance live lors de changements de composants, démarrage anticipé de l'animation Arduino, auto-réglage oscilloscope (I²C, délais Arduino), intégration scope/Bode/série/audio.

Exports : `triggerSimulation`, `stopSimulation`, `requestLiveSimulation`

led-animation.js Animation

Boucle d'animation principale (`requestAnimationFrame`) : LED depuis courbes `.tran`, runtime Arduino pas-à-pas, afficheurs 7 segments et bargraph idéaux, LCD Grove, DHT22, compteurs 74HC90 et ripple mod-10, fumée LED sur intensité, moniteur série.

Exports : `startLedAnimation`, `ensureArduinoLedAnimation`, `getAnimatedSeg7Segments`, `getIdealBargraphDisplay`, `sendUnoSerialInput...`

scope-animation.js Animation

Animation des traces CH1/CH2 de l'oscilloscope depuis plots ngspice ou traces GPIO/UART Arduino idéales ; dessin sur le canvas de l'oscillo.

Exports : `startScopeAnimation`, `drawScopeTrace`, `SCOPE_H_DIVS`

speaker-audio.js Animation · Audio

Restitution audio Web Audio API du haut-parleur simulé à partir des courbes `.tran` ou source AC interne.

Exports : `startSpeakerAudio`, `stopSpeakerAudio`

Engine/hc90-cascade.mjs Animation · Logique

Détection de chaînes 74HC90 (unités/dizaines, mod-60, reset MR) et comptage idéal pour animation ; graphe de connexions (`reachableJonctions`).

Engine/ripple-mod10.mjs Animation · Logique

Détection compteur ripple mod-10 (4× DFF + reset Q2/Q4) et animation BCD idéale pour afficheur 7 segments.

4. Moteur SPICE

Modules ESM du dossier *Simulateur/Engine/* — conversion schéma → netlist ngspice et parsing des résultats.

schematic-to-spice.mjs Moteur · Cœur

Conversion complète schéma graphique → netlist ngspice : tous composants (passifs, semi-conducteurs, logique, UNO, instruments), analyses .op, .tran, .ac, métadonnées wrdata, mode rapide Arduino sans .tran.

Exports : buildNetlistFromGraphicalState

spice-netlist-v2.mjs Moteur

Façade asynchrone de génération de deck ngspice pour le serveur Node (mode batch -b).

Exports : buildNgspiceDeck

v2/result-parser.mjs Moteur · Parsing

Parse les sorties ngspice (logs .op, fichiers wrdata tran/ac) et fusionne mesures : voltmètres, ampèremètres, LED, oscilloscope, Bode, 7 segments, bargraph, portes logiques, HC90.

Exports : mergeVoltmeterMeasurements, mergeScopePlotsFromTranWrdata, mergeBargraphFromTranWrdata... (~30 fonctions)

Logique numérique

logic-rails.mjs Logique

Niveaux logiques en tension (rails 3,3 V / 5 V, seuils, conversion tension → binaire).

logic-sequential.mjs Logique

Modèles SPICE DFF, JK, 74LS00, 74LS74, CD4511, 74HC90 ; choix XSPICE ou sources B.

logic-xspice.mjs Logique

Intégration XSPICE (d_dff, d_jkff, adc/dac, digital.cm).

logic-cd4511-bsource.mjs Logique

CD4511 par sources comportementales (sans XSPICE, compatible Linux).

logic-cd4511-xspice.mjs Logique

CD4511 via XSPICE (d_d latch, d_genlut, tables BCD→7 segments).

logic-74hc90.mjs Logique

Compteur décade 74HC90/74LS90 (ripple, broches CP/MR/MS/Q).

ngspice-xspice-probe.mjs Logique

Détecte si le binaire ngspice supporte XSPICE (devhelp d_dff).

Instruments, affichage et bus I²C

arduino-uno.mjs SPICE · UNO

Modèle SPICE Arduino UNO : rails 5 V/3,3 V/GND, GPIO DC/PULSE/PWL selon sketch, bus I²C.

bcd-seg7.mjs Affichage

Tables BCD 0–9 → segments 7 segments ; conversion tensions Q → chiffre.

voltmeter-display.mjs Instruments

Quantification affichage voltmètre (signaux logiques vs analogiques).

i2c-protocol.mjs I²C

Générateur formes d'onde I²C (100 kHz, timings UM10204) → chaîne SPICE PWL.

i2c-bus-ideal.mjs I²C

Ajoute SDA/SCL à la netlist quand LCD Grove est câblé sur UNO.

lcd-pcf8574-i2c.mjs I²C

Encode commandes LiquidCrystal_I2C (PCF8574 + HD44780 4 bits) en transactions I²C.

build-bridge-json.mjs Utilitaire CLI

Script CLI : génère un circuit JSON exemple (pont de diodes) et valide la netlist.

simulate-smoke.mjs Utilitaire CLI

Test de fumée CLI : pile + R + voltmètre → ngspice → parseur.

5. Co-simulation Arduino

arduino-sketch-parse.mjs Arduino · Cœur

Interpréteur de sketch : `pinMode`, `digitalWrite`, `delay`, registres AVR (PORTD/B/C), boucles `if/while`, expressions (`PORTD=PORTD/2`, `<<`), runtime pas-à-pas, UART série, phases GPIO temporelles pour SPICE et animation.

Exports : `parseArduinoSketch`, `createArduinoRuntime`, `stepArduinoRuntime`, `resolvePinLevelsAt`

arduino-avr-registers.mjs Arduino

Modèle registres AVR DDR/PORT pour broches D0–D13 et A0–A5.

arduino-gpio-ideal.mjs Arduino

GPIO idéal 0/5 V pour voltmètres, CD4511, 7 segments et bargraph sans relancer ngspice.

arduino-analog-ideal.mjs Arduino

Lecture analogique A0–A5 : tension réseau → ADC 10 bits (0–1023).

arduino-uart-wave.mjs Arduino

Modèle UART 8N1 (TX=D1) pour oscilloscope et moniteur série.

scope-arduino-ideal.mjs Arduino

Synthèse traces oscilloscope idéales depuis GPIO ou UART TX.

dht22-sketch-parse.mjs Arduino · Capteur

Parse sketches DHT.h : broche DATA, lectures température/humidité simulées.

dht22-ideal.mjs Arduino · Capteur

Câblage Grove DHT22 → UNO et injection valeurs dans `Serial.print`.

grove-lcd-sketch-parse.mjs Arduino · LCD

Interpréteur `LiquidCrystal_I2C` : `init`, `setCursor`, `print`, `clear`, `scroll`, phases temporelles.

grove-lcd-ideal.mjs Arduino · LCD

Affichage idéal Grove LCD 16×2 câblé sur UNO (variables analogiques/DHT).

6. Serveur et déploiement

SimulateurH/main.js Serveur · Electron

Processus principal Electron « Simulateur H » : démarre le serveur HTTP local, ouvre BrowserWindow sur l'UI (? app=h), instance unique, arrêt propre du backend.

SimulateurH/simulate-server.cjs Serveur · Express

Serveur Express autonome embarqué : fichiers statiques, POST /api/simulate (netlist → ngspice → parsing), routes Arduino, détection XSPICE, gestion digital.cm, préchargement moteur SPICE.

Exports : startStandaloneSimulateServer, createSimulateStandaloneApp

server.js Serveur · Site web

Serveur Node.js principal du site (Express) : pages publiques, authentification, chat, et route /api/simulate identique pour déploiement Render/Docker.

tools/simulate-standalone-server.cjs Serveur · Généré

Variante serveur minimal générée depuis server.js pour usage standalone (référence de build).

7. Outils Node.js (tools/)

simulate-engine-loader.cjs Outil

Cache de chargement ESM des modules moteur SPICE — évite ~25 import() par requête (performance Render et Simulateur H).

Exports : loadSimEngineModules, preloadSimEngineModules

ngspice-bundle.cjs Outil

Résolution binaire ngspice (NGSPICE, Simulateur/bin/, compatibilité Windows/Linux, chemin digital.cm).

arduino-api.cjs Outil · Arduino CLI

Wrapper arduino-cli : statut, compilation sketch UNO, recherche/installation bibliothèques.

arduino-routes.cjs Outil · API REST

Routes Express /api/arduino/* (status, compile, libraries).

build-standalone-server.cjs Outil · Build

Régénère simulate-standalone-server.cjs à partir de server.js.

check-ngspice.cjs Outil · Diagnostic

Vérifie la présence et la compatibilité du binaire ngspice sur la machine.
